OpenAI Student Collective at SNU

Workshop 02

Check In Form:



bit.ly/oaisc-check-in



가능성을 작품으로.

Codex로 기능 하나를 고치고,
GitHub에 변경 이력 남기기

OpenAI Student Collective

The OpenAI Student Collective brings students together to experiment with the newest AI tools through workshops, challenges, and campus events.





Meet your Campus Leads



TAEHYUN KIM
전기정보공학부 25학번

물리천문학부 복수전공

Favorite fun use-case:
Building actual services using AI

SEOLYNN PARK
영어영문학과 26학번

Favorite fun use-case:
Building apps with Codex

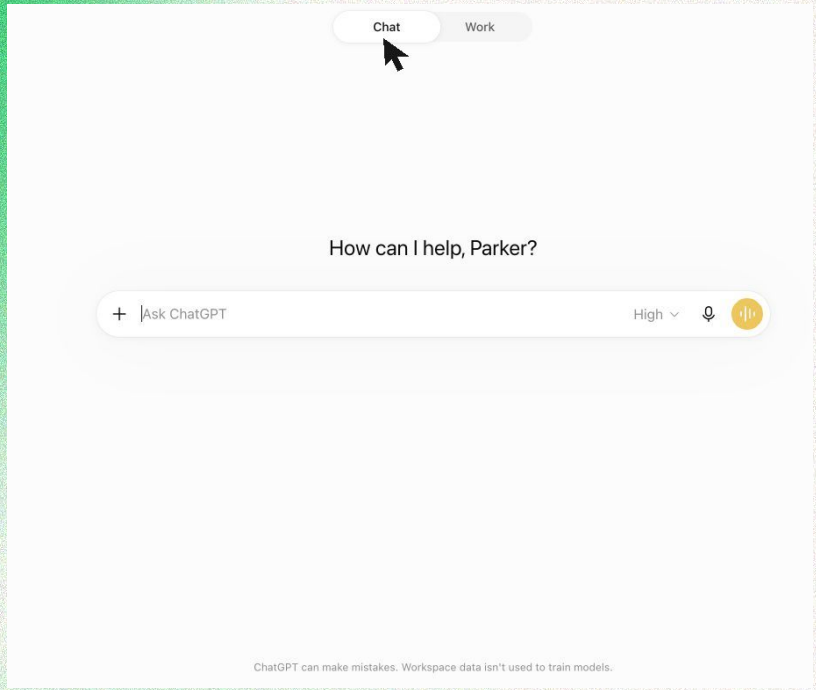
Form a group

Get into groups of 2–3 with people nearby. You'll work together throughout today's workshop.

- Introduce yourself
  - Name
  - Major
  - Year
  - Hometown
- Discuss: What's something you've been unusually into lately?

작업 도구

ChatGPT로 요청을 정리하고,
Codex로 코드를 수정합니다.



요청 구체화

바꿀 기능과 완료 조건을
정합니다

수정하고 확인

Codex로 수정하고
브라우저에서 눌러봅니다

변경 기록

Git으로 commit, GitHub로
push합니다

Activity

Codex로 기능 하나를 수정하고, 직접 확인한 뒤 GitHub에 기록합니다.
2–3명이 요청·조작·검증 역할을 번갈아 맡아보세요.

오늘은, 내 앱을 고치고 기록합니다

내 프로젝트에서

로컬에서 실행하기
기능 하나 바꾸기

작동 확인 → 변경 기록 → 복구까지 경험합니다.

오늘의 완성 기준

기능 수정 1개

commit · push · revert

공통 예제

모임 카드에 즐겨찾기 버튼을 추가합니다.

같은 모임 카드, 버튼 하나의 차이

수정 전

모임 이름·시간·장소
마음에 드는 모임을 기억하기
어려움

시작본과 완성본을 직접 열어 비교합니다.

수정 후 · 예시

☆ 즐겨찾기

클릭 → ★ 선택됨

한 번 더 클릭하면?

선택이 해제됩니다. 다른 카드의 상태는 유지됩니다.

실제 앱에서 전후를 확인해봅시다

관찰할 것

어느 화면이 바뀌었나요?
클릭하면 무엇이 달라지나요?

같은 예제에서 수정 전후를 직접 비교합니다.

openAI_WS · 실습 자료

시작본 / 완성본 열기 ↗

README · 사전 준비 안내 ↗

60초 라이브 시연

A·B 선택 → A만 해제 → 새로고침으로 초기화 확인

파일 · 터미널 · 브라우저의 역할

VS Code에서 확인할 세 곳

파일: 앱의 코드와 설정을 봅니다.

터미널: 실행 명령과 오류를 확인합니다.

브라우저: 바뀐 기능을 직접 눌러봅니다.

확인하는 순서

파일 열기

↓

명령 실행

↓

화면 확인

같은 프로젝트 폴더에서 편집하고 실행하는지 확인하세요.

먼저, 내 컴퓨터에서 앱을 열어요

프로젝트 폴더 열기

openAI_WS 폴더를 엽니다.
새 터미널에서 실행합니다.

Node.js 사전 설치 · 별도 패키지 설치 없음

openAI_WS 폴더에서 실행

```
npm run dev
```

http://localhost:5173

시작본 선택 → 수정 파일 확인

workshop2/starter/를 수정합니다. 내 앱은 해당 README를 따르세요.

Git과 GitHub는 역할이 달라요

Git · 내 컴퓨터

코드의 변경 이력을 관리합니다.
이전 변경을 확인하고
되돌립니다.

파일 저장과 commit은 다릅니다. commit과 push도 구분하세요.

GitHub · 온라인 저장소

기록 공유

push → 원격에 반영

오늘 사용할 흐름

수정 → 확인 → stage → commit → push

기능과 완료 조건을 함께 요청해요

Codex에 요청할 내용

starter의 모임 카드에 즐겨찾기를 추가해줘.
클릭하면 선택, 다시 누르면 해제해줘.
카드별로 독립적으로 동작하게 해줘.
새로고침하면 초기화해. 저장소는 쓰지 마.
기존 모임 정보와 레이아웃은 유지해줘.
수정 파일과 확인 방법을 알려줘.

완료 조건

01 선택·해제되나요?

02 다른 카드는 유지되나요?

03 새로고침 시 초기화되나요?

화면 안에서만 상태를 유지합니다. DB 저장은 WS3에서 다룹니다.

요청 후에는 바뀐 파일을 확인해요

진행자가 보여줄 것

기능 요청을 전달합니다.
변경된 파일 목록을 펼쳐봅니다.
추가·삭제된 코드를 비교합니다.

요청하지 않은 변경이 있으면
그 이유를 묻고 범위를 조정합니다.

AI와 작업하는 순서

요청하기
↓
변경 읽기
↓
직접 실행

AI가 완료했다고 말해도, 브라우저에서 직접 확인합니다.

버튼을 눌러 완료 조건을 확인해요

같은 순서로 확인하기

A 카드 선택 → 선택 표시 확인
B 카드 선택 → A·B 모두 선택
A 다시 클릭 → A만 해제

새로고침 → 모두 초기화
카드 정보·기존 동작도 확인

기존 기능도 확인

01 카드 내용이 보이나요?

02 다른 기능도 작동하나요?

03 오류가 새로 생겼나요?

화면이 바뀐 것과 기능이 작동하는 것을 함께 확인합니다.

확인한 변경을 commit · push

기록을 남기는 네 단계

Source Control에서 변경을 봅니다.
이번 기능의 파일만 stage합니다.
이유가 드러나는 메시지로 commit합니다.

push 후 GitHub에서 같은 기록이
보이는지 확인합니다.

기록의 흐름

변경 파일

↓

로컬 commit

↓

GitHub

예시 메시지: 모임을 다시 찾기 쉽게 즐겨찾기 버튼 추가

내 프로젝트에 맞는 요청문

[화면]에서 [행동]을 하면
[결과]가 나오도록 수정해줘.

기존 [유지할 기능]은 그대로 두고,
완료 조건과 확인 방법을 알려줘.

예시

스터디 카드에서 버튼을 누르면
관심 표시를 켜고 끄게 해줘.

버튼 · 문구 · 화면 중 오늘 끝낼 수 있는 변경 하나를 고르세요.

이제, 내 앱에 직접 적용해봅시다

23분

기능 수정 1개
직접 눌러 작동 확인

22–27

요청 구체화

27–40

수정·실행

40–45

완료 조건 확인

오류는 재현 순서와 함께 알려줘요

Codex에 전달할 오류 설명

한 행동: [어디서 무엇을 했는지]
기대한 결과: [어떻게 되어야 하는지]
실제 결과: [무엇이 일어났는지]

오류 문구와 관련 화면을 첨부할게.
원인을 설명하고 작은 수정부터 제안해줘.

확인 자료

01 실행한 명령

02 오류 메시지 원문

03 다시 재현하는 순서

막히면 운영진에게 요청하세요. 설치 문제로 실습 전체를 멈추지 않습니다.

이번 변경에 필요한 파일만 골라요

Source Control

파일을 눌러 변경 내용을 봅니다.
이번 기능에 필요한 파일을 stage합니다.

비밀키나 개인 설정 파일이 포함되지 않았는지도 확인합니다.

commit 전에 확인

변경 범위

요청한 기능과 일치?

관계없는 파일이 있다면

이번 commit에서 제외하고, 왜 바뀌었는지 확인합니다.

무엇을 왜 바꿨는지 남깁니다

commit

stage한 변경을 기록합니다.
무엇을 왜 바꿨는지
메시지로 남깁니다.

commit 후 기록 목록에서 방금 만든 메시지를 확인하세요.

메시지 예시

즐겨찾기 추가

모임을 다시 찾기 쉽게

내 변경을 한 줄로

[사용자 목적]을 위해 [기능]을 바꿨다.

push 후 GitHub에서도 확인해요

원격 저장소에 반영하기

origin이 본인 Fork인지 확인합니다.
push로 로컬 commit을 전송합니다.

GitHub에서 같은 브랜치를 엽니다.
방금 만든 commit 메시지와
변경 파일이 보이는지 확인합니다.

완료 확인

01 올바른 저장소인가요?

02 같은 브랜치인가요?

03 방금 기록이 보이나요?

push가 거절되면 오류를 확인하고 도움을 요청하세요. 강제 push는 하지 않습니다.

실수도 기록을 남기며 되돌립니다

revert

잘못된 commit의 변경을 취소하는 새 commit을 만듭니다.

복구 실습은 별도 로컬 저장소에서 진행합니다.

복구 연습 안내

RECOVERY.md 열기 ↗

사전 준비: 별도 연습 폴더 생성

복구 전 준비

recovery-practice 폴더에서, 버튼을 비활성화한 최신 commit을 확인합니다.

대상 commit을 확인하고 되돌려요

연습 저장소에서 실행

```
git status

git log --oneline -2

git revert --no-edit HEAD
```

새로고침하여 버튼 동작을 확인합니다.

실행 전 확인

01 연습 저장소인가요?

02 작업 폴더가 깨끗한가요?

03 대상 기록이 맞나요?

이 예제의 최신 오류 commit만 되돌립니다. 충돌 시 운영진에게 요청하세요.

새 기록과 화면을 함께 확인해요

되돌린 뒤 확인하기

버튼이 다시 동작하는지 봅니다.
기존 두 commit이 남아 있는지 봅니다.
새 Revert commit이 추가됐는지 봅니다.

이 연습은 로컬 복구까지 진행합니다.
GitHub push는 본인 앱에서 확인합니다.

이력은 이어집니다

기존 변경

↓

복구 commit

↓

복구된 동작

revert는 과거 기록을 지우는 작업이 아닙니다.

더 해보고 싶다면: branch와 PR

별도 branch에서 변경을 만듭니다.
commit하고 해당 branch를 push합니다.

Pull Request로 변경 이유와
확인한 결과를 공유합니다.

확장 과제

기본 과제를 마친 참가자만 진행.
다른 사람에게 변경 검토 요청하기.

오늘의 필수 과제는 기능 수정 · commit · push · 복구입니다.

내 앱의 변화를 30초로 소개해주세요

“[이유] 때문에 [기능]을 바꿨고,
직접 눌러 [결과]를 확인했습니다.”

변경 전후 화면과 commit 기록을 보여주세요.
저장소 링크와 바꾼 기능 한 줄을 남겨주세요.

다음 WS3: 데이터를 저장하고, 배포 링크로 공유하기

오늘의 변화 

Check out

That’s it, thanks for coming!

Scan to share what you made with the OpenAI team and get a custom graphic:

bit.ly/oaisc-check-out

How to stay connected

Come to studio hour

→ 일시·장소 추후 공지

Join the campus community

→ https://open.kakao.com/o/pqDgx8Pi

Subscribe to OpenAI Student Collective on Substack

→ www.openaistudentcollective.substack.com

Thanks for Coming!